

PM Controller

Phase 1 — Completion record · schema, auth, admin setup, row-level security

Owner Apex Plumbing and Mechanical Services SC **Pilot** Paulson Cheek Mechanical

Merged PR #1 · 21 September 2026 **Status** Milestone met

1. The milestone, and how it was proven

"An admin creates a Lead by email, and that Lead sees only his project and cannot read the receipts tables at all."
That is a claim about the database, so it was tested against the database — as the role a phone actually holds (authenticated , with a real signed-in user id), never as the owner, who bypasses every policy. **56 of 56 assertions pass.**

AREA	TESTS	WHAT IS ACTUALLY ASSERTED
Project isolation	9	A Lead sees exactly one project. A sub's Lead on another job sees none of the first job's tasks. The same man is Foreman on one job and Super on another.
Material access	15	Lead reads zero receipts, zero line items, zero categories. A Foreman cannot reach the other Foreman's receipt by id. PM, Assistant PM and Manager read everything.
Sign-off	4	Lead, Foreman and Assistant PM refused. Super allowed.
History	7	An Accomplished task cannot be reopened by anyone, the Manager included. <code>task_events</code> cannot be rewritten even by the service role.
Change orders	4	Assistant PM and Super refused in both directions. PM allowed.
Locked statuses	5	A seventh status is impossible. <code>rework_count</code> cannot be typed, only derived.
Lead movement	4	Moves his own task only; cannot create, kick back, or touch another Lead's.
People & cost	8	A Lead cannot read <code>burdened_rate</code> at all. Nobody can be deleted — deactivation is the only path.

The gate was verified to actually gate. Introducing one deliberate leak — a Foreman able to see another Foreman's receipts — fails five assertions and exits non-zero. A test suite that has never been seen to fail is not evidence of anything.

2. Six defects found in the reviewed schema

Four of them would have stopped the app working entirely. Each is marked [FIX-n] at the place it was fixed; the full write-up is in docs/DECISIONS.md .

REF	DEFECT	EFFECT IF IT HAD SHIPPED
FIX-6	The audit trigger was blocked by its own table's row-level security	No status change saves, for any role, with no error pointing at the cause
FIX-1	<code>citext</code> created after the table that uses it	Migration could not apply at all

FIX-3	<code>auth.uid()</code> is null for service-role writes, but <code>actor_id</code> is NOT NULL	Every seed and background status change fails
FIX-4	The rework counter trips the guard protecting its own column	No kick-back could ever be recorded
FIX-5	Cost view unusable as <code>security_invoker</code> under column grants	Nobody sees burdened rates, PM included
FIX-2	<code>qty_needs_unit</code> ignored <code>qty_installed</code>	Close-out records "340" with no unit

3. Decision log — how Phase 1 actually went

A record of each request and what came of it. Summarised rather than transcribed, so it stays readable; the commit history and `docs/DECISIONS.md` carry the detail.

Build it as an Apple app, not a website.

SPEC §9 changed from React PWA to native SwiftUI. Backend untouched — the schema and policies never depended on the client. The three reasons were risks the Phase 0 plan had already flagged: push notifications reaching a Lead with the app closed, camera in one tap, and a close-out surviving no signal.

Name it PM Controller. Separate it from Apex. Tablet first. Add CI.

Renamed throughout. Demo data de-branded. `TARGETED_DEVICE_FAMILY "1,2"` — one binary, layouts diverging by size class. Three CI jobs added: row-level security, an iPad build, and screenshots of the app running.

Can I see results before paying for a developer account?

Yes — the \$99 is only needed to put a build on someone else's device. The real prerequisite was a Mac, and CI covers most of it: GitHub's macOS runners are free on public repositories, so every push now photographs the app on an iPad.

Go as old as iOS 24.

Raised as a factual error: there is no iOS 24, and no 19 through 25. Apple went from 18 straight to 26 in 2025. Read as "the 2024 release" and set to iOS 18 — which also reverted a costly detour to iOS 16 made an hour earlier.

Apex owns the app. Make sure it's in the code.

Reversed the earlier separation. Ownership now asserted in `LICENSE`, a copyright header on all 17 source files, and `NSHumanReadableCopyright` so it ships inside the binary.

Use the full legal name. iOS 18 works.

Apex Plumbing and Mechanical Services SC everywhere. Two things a plain replace missed: the name wrapped across a line in `LICENSE` exactly where the "Apex" shorthand is defined, and the documents were still quoting a bundle identifier that had changed.

apexmech.com is ours. Merge it.

Bundle identifier `com.apexmech.pmcontroller` confirmed correct. PR #1 merged with a merge commit rather than a squash — three of its six commits record CI failures and their causes, and on a project whose own principle is that the audit trail is the product, discarding that would be the wrong instinct.

Three CI failures, none of them flakes

FAILURE	CAUSE
Incompatible device	The simulator picker chose a 2017 iPad Pro that iOS 26.5 will not run. Having a device type and a runtime does not mean they pair. Now reads each runtime's published <code>supportedDeviceTypes</code> .
Generic parameter could not be inferred	<code>@EnvironmentObject</code> needs an explicit type where <code>@Environment(Session.self)</code> infers one. Introduced by the iOS 16 conversion that was later reverted.
Missing bundle ID	Supplying <code>INFOPLIST_FILE</code> means Xcode synthesises nothing, so <code>CFBundleIdentifier</code> was simply absent and the built app could not be installed.

4. What is verified, and what is not

VERIFIED	NOT VERIFIED
• Both migrations apply to a real Postgres 16 • 56/56 policy assertions, run as a signed-in user • The suite fails loudly when a rule regresses • The Swift compiles for iPad at the iOS 18 floor • The app launches and renders every Phase 1 screen	• The Edge Function has never been deployed • No migration has run against a live Supabase project • Nobody has signed in with a real credential • Behaviour on a physical iPad — CI carries only recent simulator runtimes • Push notifications (Phase 5) entirely

5. Carried into Phase 2

☒ App named and locked — PM Controller, <code>com.apexmech.pmcontroller</code>
☒ Ownership asserted in the code
☒ Deployment floor settled at iOS/iPadOS 18
☒ A Mac is available; no Apple Developer Program needed yet
☒ CI gates the access rules on every push
☐ Create the Supabase project and apply <code>0001</code> , <code>0002</code> , then seed
☐ Deploy <code>admin-create-user</code> and create the first real Manager
☐ Open it in Xcode and run it — the first thing CI cannot do for you
☐ Data-ownership terms with PCM in writing before any of its data is entered
☐ Confirm which iPads the crews carry — anything below iPadOS 18 is out
☐ Burdened rates: real figures, or placeholder multipliers

The one thing to watch. Everything proven so far is proven against a database on a test machine. The first real sign-in, against a real Supabase project, is where Phase 1 stops being a claim. It is a fifteen-minute job and it is worth doing before Phase 2 gets long.

6. Ideas worth keeping

IDEA	WHY IT CAME UP
Tool disposition flag	The Phase 0 plan had a job-charged vs. returns-to-shop flag on tools that SPEC §6 dropped. It is a real source of quiet job overrun and belongs in Phase 6d.
Screenshots as a review artifact	Built to work around not having a Mac; worth keeping permanently. Every push produces a visual record of what the app looked like on that commit.
The seed data doubles as preview data	Preview mode mirrors <code>seed.sql</code> , including the man who is Super on one job and Foreman on another. It keeps screenshots honest and makes the per-project role rule visible in every one of them.

The test suite is a sales asset

"A Foreman cannot see another Foreman's receipts, and here is the test that proves it" is a stronger claim to a pilot customer than any feature list.
